

MEDIATEK

MT8676 Android Suspend and Resume User Manual

Version: 1.0
Release date: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-08-12	Xiaojun Zheng	Official release

Table of Contents

Version History 2

Table of Contents 3

List of Figures 3

List of Tables 3

1 Suspend/Resume 4

1.1 Overview 4

1.2 Architecture/Process Overview 4

1.2.1 Android Suspend/Resume Flow 4

1.2.2 Kernel Suspend/Resume Flow 5

1.3 Configuration/Customization Guideline 7

1.4 Frequently Asked Questions/Troubleshooting 7

1.4.1 How to Determine if the System Has Successfully Entered Sleep Mode 7

1.4.2 How to Identify Wakeup Sources 7

1.4.3 Wakeup Source Analysis 8

1.4.4 How to Analyze Issues with Failing to Enter Sleep Mode 9

1.4.5 How to Analyze High Power Consumption During Sleep Mode 10

Exhibit 1 Terms and Conditions 11

List of Figures

Figure 1-1. Android Suspend/Resume flow 4

Figure 1-2. Android Suspend/Resume source code 5

Figure 1-3. Kernel Suspend/Resume flow 6

Figure 1-4. Wakelock dump 10

List of Tables

Table 1-1. MT8676 wakeup source list 8

1 Suspend/Resume

1.1 Overview

This chapter mainly introduces the MT8676 Suspend/Resume process and common troubleshooting methods.

On the MT8676 platform, Suspend refers to suspend to RAM, a state where all devices enter a low-power mode, with only RAM self-refresh remaining.

Suspend Power State: The entire system is in standby, with very low power consumption.

- Tasks “frozen”
- External devices/internal modules powered off or in low-power mode
- System PLL/clock close
- ARM off
- DRAM self-refresh
- PMIC in low-power mode
- Vcore off
- SPM running
- Wait for HW wakeup event

1.2 Architecture/Process Overview

1.2.1 Android Suspend/Resume Flow

The Android system’s Suspend process is mainly divided into two major stages. The first stage is screen-off, which means turning off the display and entering light sleep. After the system enters light sleep, if there is no wakelock, it will trigger the Kernel Suspend process. The resume process is the opposite. Refer to [Figure 1-1](#).

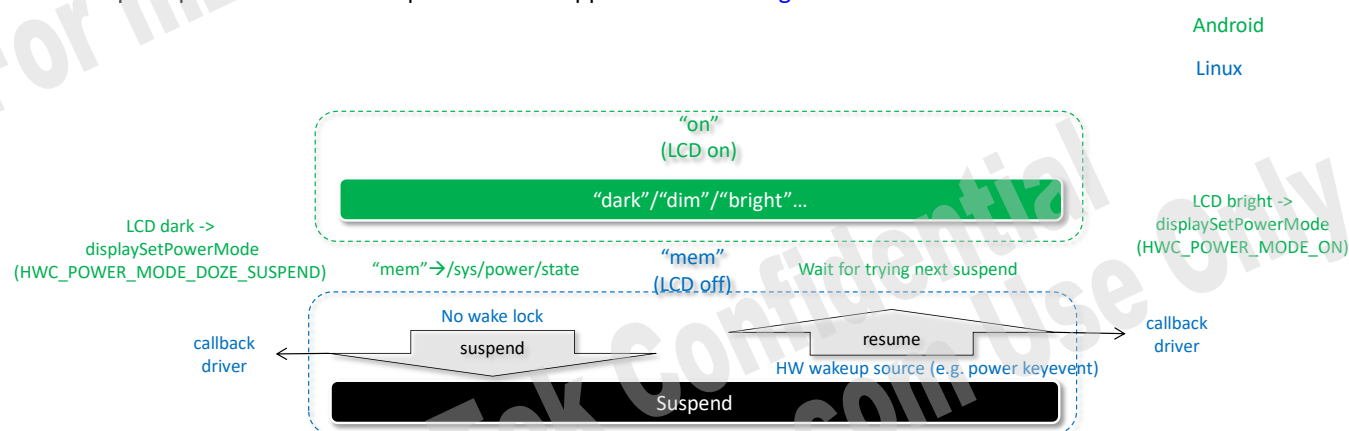


Figure 1-1. Android Suspend/Resume flow

The corresponding source code is as follows.

```

void SystemSuspend::initAutosuspendLocked() {
    if (mAutosuspendThreadCreated) {
        LOG(INFO) << "Autosuspend thread already started.";
        return;
    }

    std::thread autosuspendThread([this] {
        auto autosuspendLock = std::unique_lock(mAutosuspendLock);
        bool shouldSleep = true;

        while (true) {
            if (!mAutosuspendEnabled) {
                mAutosuspendThreadCreated = false;
                return;
            }
            // If we got here by a failed write to /sys/power/wakeup_count; don't sleep
            // since we didn't attempt to suspend on the last cycle of this loop.
            if (shouldSleep) {
                mAutosuspendCondVar.wait_for(autosuspendLock, mSleepTime,
                    [this] { return !mAutosuspendEnabled; });
            }

            if (!mAutosuspendEnabled) continue;

            string wakeupCount;
            {
                autosuspendLock.unlock();

                lseek(mWakeupCountFd, 0, SEEK_SET);
                wakeupCount = readFd(mWakeupCountFd);

                autosuspendLock.lock();
            }

            if (wakeupCount.empty()) {
                PLOG(ERROR) << "Error reading from /sys/power/wakeup_count";
                continue;
            }

            shouldSleep = false;

            mAutosuspendCondVar.wait(
                autosuspendLock, [this] { return mSuspendCounter == 0 || !mAutosuspendEnabled; });
            // The mutex is locked and *MUST* remain locked until we write to /sys/power/state.
            // Otherwise, a WakeLock might be acquired after we check mSuspendCounter and before we
            // write to /sys/power/state.

            if (!mAutosuspendEnabled) continue;

            if (!hasAliveAutosuspendTokenLocked()) {
                disableAutosuspendLocked();
                continue;
            }

            if (!WriteStringToFd(wakeupCount, mWakeupCountFd)) {
                PLOG(VERBOSE) << "Error writing from /sys/power/wakeup_count";
                continue;
            }
            bool success = WriteStringToFd(kSleepState, mStateFd);
            shouldSleep = true;

            {
                autosuspendLock.unlock();

                if (!success) {
                    PLOG(VERBOSE) << "Error writing to /sys/power/state";
                }

                struct SuspendTime suspendTime = readSuspendTime(mSuspendTimeFd);
                updateSleepTime(success, suspendTime);

                std::vector<std::string> wakeupReasons = readWakeupReasons(mWakeupReasonsFd);
                if (wakeupReasons == std::vector<std::string>({kUnknownWakeup})) {
                    LOG(INFO) << "Unknown/empty wakeup reason. Re-opening wakeup_reason file.";

                    mWakeupReasonsFd =
                        std::move(reopenFileUsingFd(mWakeupReasonsFd.get(), 0, O_CLOEXEC | O_RDONLY));
                }
                mWakeupList.update(wakeupReasons);
                mControlService->notifyWakeup(success, wakeupReasons);

                // Take the lock before returning to the start of the loop
                autosuspendLock.lock();
            }
        }
    });
    autosuspendThread.detach();
    mAutosuspendThreadCreated = true;
    LOG(INFO) << "automatic system suspend enabled";
}

```

Figure 1-2. Android Suspend/Resume source code

Main flow description:

1. **sleep mSleepTime:** The initial value is 100 ms. This value will change depending on whether the suspend operation is successful or not.
2. **Read the value of /sys/power/wakeup_count:** If there is an active wakeup source at this time, the process will be blocked in the kernel.
3. **Return wakeup_count:** This happens when there are no active wakeup sources.
4. **Write back wakeup_count:** If the write-back is successful, the process continues.
5. **Write mem to /sys/power/state:** This initiates the Kernel Suspend process. If successful, the process will halt at this line and wait for the Kernel Resume to continue execution. If the suspend fails, mSleepTime will double, and the loop will retry the suspend process.

1.2.2 Kernel Suspend/Resume Flow

The flow of Kernel Suspend/Resume is shown in [Figure 1-3](#).

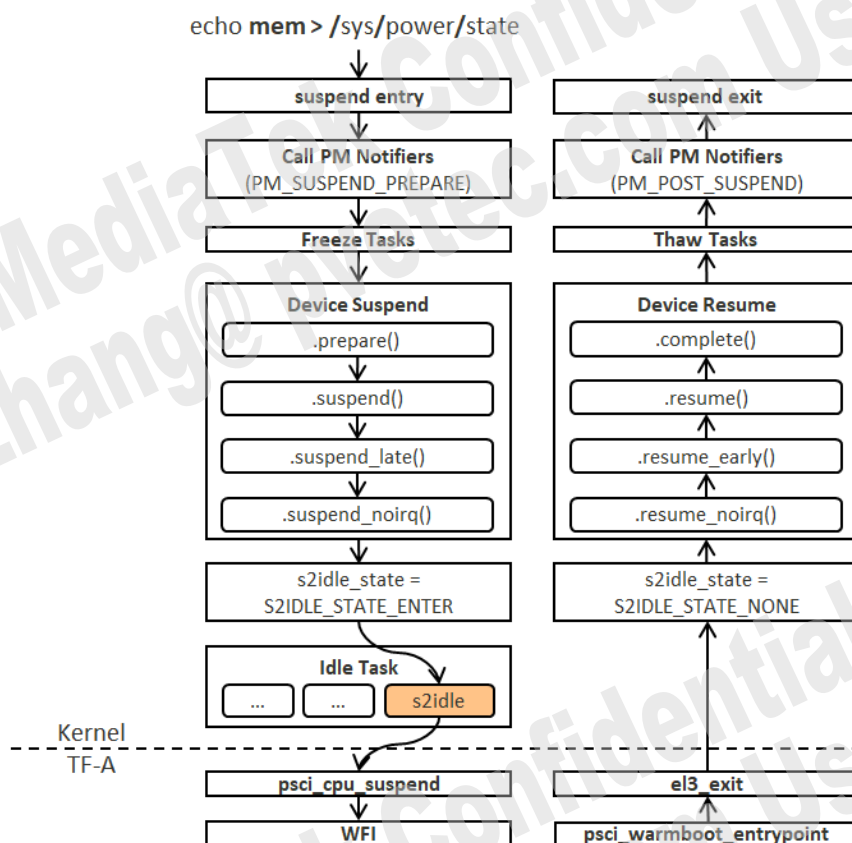


Figure 1-3. Kernel Suspend/Resume flow

When the upper layer writes mem to /sys/power/state, it triggers the Kernel Suspend process.

1. Process Freeze:

- First, user space processes are frozen.
- Next, kernel threads are frozen.
- If a wakelock is detected during this step, the suspend process will be aborted. If the freeze is successful, the suspend process continues.

2. Device Suspend Flow:

- The system calls the registered `prepare`, `suspend`, and `suspend_late` callbacks for each device in sequence. This completes the necessary preparations for each device driver during the sleep phase.

3. Enter s2idle:

- The system enters `s2idle_enter()`, setting `s2idle_state` to `S2IDLE_STATE_ENTER`: All CPUs enter the idle loop.

4. Enter TF-A psci_cpu_suspend:

- The system enters the Trusted Firmware-A (TF-A) `psci_cpu_suspend`.

5. Power Down Unused Hardware:

- Unused hardware is powered down, and the entire system enters low-power sleep mode.

After entering the sleep mode, if there is a wakeup request from a wakeup source, the system will wake up and enter the resume process. The resume process is the reverse of the suspend process.

1.3 Configuration/Customization Guideline

Since the Suspend/Resume flow is largely based on the native Android and Kernel flow, there are not many custom settings required.

1.4 Frequently Asked Questions/Troubleshooting

1.4.1 How to Determine if the System Has Successfully Entered Sleep Mode

Screen-off might only indicate that the system has entered light sleep, which does not necessarily mean that it has successfully entered deep sleep. To determine if the system has successfully entered suspend mode, you need to check the kernel log. If the system has successfully entered suspend mode, the kernel log will stop printing.

To enable more detailed debug log information for determining if the system has successfully entered suspend mode, you need to run the following commands.

```
adb shell "echo 8 8 8 8 > /proc/sys/kernel/printk"
adb shell "echo 1 > /sys/module/kernel/parameters/initcall_debug"
adb shell "echo 1 > /proc/mtprintk"
```

In the kernel log, the following keywords indicate the progress of the suspend process.

1. "PM: Syncing filesystems ...":
This indicates that the kernel has started the suspend process.
2. "suspend of devices complete after xxx msecs":
This indicates that the device suspend phase has been completed.
3. "late suspend of devices complete after xxx msecs":
This indicates that the late suspend phase for devices has been completed.
4. "noirq suspend of devices complete after xxx msecs":
This indicates that the noirq (no interrupt request) suspend phase for devices has been completed.
5. "suspend enter":
This indicates that the suspend process is complete and the system has entered the suspend state.

1.4.2 How to Identify Wakeup Sources

Search for the keyword "suspend wake up by" in the kernel log to identify the wakeup source.

e.g.

Pwrkey Wakeup (and other EINT Wakeups):

```
[SPM] suspend wake up by R12_EINT_EVENT_B, timer_out = 207308
```

To determine which specific EINT caused the wakeup, first look for the log entry:

```
EINT xxx is pending
```

Then, use the following command to see which interrupt corresponds to xxx:

```
cat /proc/interrupts
```

Modem-Related Wakeup:

```
[SPM] suspend wake up by R12_CCIF0_EVENT_B, timer_out = 1825253
```

Timer (PCM_Timer) Wakeup:

```
[SPM] suspend wake up by PCM_TIMER, timer_out = 65612
```

1.4.3 Wakeup Source Analysis

Table 1-1 is a list of wakeup sources supported by the MT8676 platform.

Table 1-1. MT8676 wakeup source list

Name	Control Bit	Description	Can it be Disabled
R12_PCM_TIMER	0	Timer event for its timeout setting	Y
R12_KP_IRQ_B	2	Keypad pressed/released action been detected	Y
R12_APWDT_EVENT_B	3	RGU wakes up CPU in suspend mode.	Y
R12_APXGPT1_EVENT_B	4	AP GPT timer timeout event	Y
R12_CONN2AP_SPM_WAKEUP_B	5	Connectivity IC(Wi-Fi/BT/GPS) event	Y
R12_EINT_EVENT_B	6	EINT event	N
R12_CONN_WDT_IRQ_B	7	Connectivity IC watchdog timeout event	Y
R12_CCIF0_EVENT_B	8	MD to AP CCIF wakeup event	Y
R12_CCIF1_EVENT_B	9	MD to AP CCIF wakeup event	Y
R12_SSPM2SPM_WAKEUP_B	10	SSPM event	Y
R12_SCP2SPM_WAKEUP_B	11	SCP sensor event	N
R12_ADSP2SPM_WAKEUP_B	12	ADSP event	Y
R12_USBX_CDSC_B	14	USB event	Y
R12_USBX_POWERDWN_B	15	USB remote wakeup	Y
R12_SYS_TIMER_EVENT_B	18	System timer	N
R12_EINT_EVENT_SECURE_B	19	EINT event	N
R12_SCP_CIRQ_IRQ_B	22	SCP_CIRQ wakeup event	Y
R12_MD2AP_PEER_EVENT_B	23	MD event	Y
R12_MD1_WDT_B	25	MD1 watchdog timeout	Y
R12_REG_CPU_WAKEUP_B	28	Internal Wakeup	N
R12_APUSYS_WAKE_HOST_B	29	APUSYS event	Y
R12_PCIE_WAKEUP_EVENT_B	30	PCIe event	Y
R12_MSDC_WAKEUP_EVENT_B	31	MSDC event	Y

The wakeup sources in the list can be disabled through the following way, but only the wakeup sources marked as“Y” in the “Can it be Disabled” column can be disabled.

You can quickly disable the desired wakeup sources using the system-provided debug command.

1. Use the following command to read the current system wakeup source settings.

```
cat /proc/mtk_lpm/power/suspend_ctrl | grep -i wake_src
```

2. In [Table 1-1](#), find the control bit of the wakeup source you want to disable in the “Control Bit” column. Then, set the corresponding bit of the read wake_src to 0, and finally use the following command to write the modified wake_src back into the system (do not modify the control bits of wakeup sources you do not want to disable):

```
echo wake_src 0x***** > /proc/mtk_lpm/power/suspend_ctrl
```

3. If you want to re-enable a wakeup source, set the corresponding control bit back to 1.

1.4.4 How to Analyze Issues with Failing to Enter Sleep Mode

If the system fails to enter sleep mode after the screen turns off, you can use the following commands to determine which wakelock is preventing the system from entering sleep mode.

- Check which modules in userspace are holding wakelocks.

dumpsys power

e.g.

```
Wake Locks: size=1
  SCREEN_DIM_WAKE_LOCK          'CAR.POWER' ACQ=-5m14s440ms (uid=1000 pid=1467)

Suspend Blockers: size=4
  PowerManagerService.WakeLocks: ref count=1
  PowerManagerService.Display: ref count=1
  PowerManagerService.Broadcasts: ref count=0
  PowerManagerService.WirelessChargerDetector: ref count=0

Display Power: state=ON
```

- Check which modules in the kernel are holding wakelocks.

```
cat /sys/kernel/debug/wakeup_sources
```

Observe the 5th column “active_since” in the output. The number that is non-zero and continuously increasing indicates the wakelock that is preventing the system from entering standby mode.

For example, in [Figure 1-4](#), the USB wakelock is blocking the system from entering standby mode.

```
spm0675p1_64:/ # cat /sys/kernel/debug/wakeup_sources
```

name	active_count	event_count	wakeup_count	expire_count	active_since	total_time	max_time	last_change	prevent_suspend_time
event2	0	0	0	0	0	0	0	0	0
event1	8	8	0	0	0	203	145	41989	0
event3	0	0	0	0	0	0	0	0	0
event0	0	0	0	0	0	0	0	0	0
inotify	20	20	0	0	0	4	1	130596	0
eventpoll	2	2	0	0	0	0	0	31562	0
(timerfd)	30	30	0	0	0	0	0	130596	0
(timerfd)	18	18	9	0	0	1223	151	41937	0
(timerfd)	0	0	0	0	0	0	0	0	0
(timerfd)	18	18	4	0	0	575	151	121306	0
(timerfd)	20	20	2	0	0	264	131	37344	0
(timerfd)	0	0	0	0	0	0	0	0	0
(timerfd)	0	0	0	0	0	0	0	0	0
eventpoll	44	56	0	0	0	0	0	121306	0
gpswakelock	0	0	0	0	0	0	0	0	0
bt_drv	0	0	0	0	0	0	0	0	0
wmtFuncCtrl	3	3	0	0	0	2872	1456	43697	0
AT662x	7	7	0	0	0	374	72	52259	0
rpa1	0	0	0	0	0	0	0	0	0
rpa0	0	0	0	0	0	0	0	0	0
usb-hdrc	2	2	0	0	107302	134244	107302	41949	0
mdla	0	0	0	0	0	0	0	0	0
sibc	0	0	0	0	0	0	0	0	0

Figure 1-4. Wakelock dump

The command needs to be entered via UART because plugging in a USB device can prevent the system from entering standby mode.

1.4.5 How to Analyze High Power Consumption During Sleep Mode

- First check for frequent wakeup issues. If there are frequent wakeup issues, please refer to [Section 1.4.2](#) to identify the wakeup sources.
- If the system successfully enters sleep mode but the power consumption is still high, first check for any peripherals that may not be properly powered down.
- If the high power consumption is traced to the MediaTek SoC, please provide logs to MediaTek for further analysis.

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.